KiCad: dallo schema al footprint

Lezione 1 — 2MAK, SST-MEC-25c — Introduzione al CAD elettronico

17/09/2026

1 Cos'è KiCad e a cosa serve

KiCad è un software **CAD** (Computer-Aided Design) che permette di disegnare **schemi elettrici** e **circuiti stampati**. Da sempre, il **processo di progettazione** con questi strumenti si divide in due parti distinte:

- lo **schematico**: lo **schema elettrico** che mostra le **connessioni logiche** tra i componenti;
- il **PCB** (Printed Circuit Board): il **disegno fisico** del circuito stampato che verrà effettivamente prodotto.

Nota

Le **connessioni** tra i componenti sullo schema sono **indipendenti** dalla loro **posizione fisica** sul disegno: quello che conta è ***cosa è collegato a cosa***, non dove disegniamo graficamente i simboli.

2 Come KiCad memorizza lo schema: la netlist

È importante che il software **identifichi** in modo **univoco** ogni componente: se ci sono due **resistenze**, vanno **numerate** (R1, R2, ...). Ogni componente ha dei **terminali** (pin): una resistenza R1 ha, ad esempio, **`R1.pin1`** e **`R1.pin2`**.

Internamente, KiCad non memorizza il disegno come un'**immagine artistica** (come farebbe un programma di grafica generico): scrive un **file di testo** che elenca le **connessioni**, ad esempio "`R1.pin2` è collegato a `R2.pin1`".

Definizione

Questo file che elenca **tutte le connessioni** si chiama **netlist**. Ogni software di progettazione elettronica genera un disegno e produce una netlist – l'**elenco completo** di tutte le connessioni che devono esistere nel circuito finale.

Nota

Fino a KiCad 7 la netlist andava **generata manualmente**; dalla versione 8 in poi il passaggio è **automatico** e nascosto all'utente. Capire comunque cosa succede "sotto" aiuta a ragionare sul processo – ed è un **file di testo**, quindi in linea di principio potrebbe anche essere generato o **modificato** da un programma, ad esempio da un'**intelligenza artificiale** capace di scrivere codice (anche se il risultato non avrebbe comunque una **rappresentazione grafica** leggibile per un umano: KiCad la ricostruirebbe comunque a partire dal file).

Esempio

Esiste un progetto chiamato **Flux** (Flux.ai) che punta esattamente in questa direzione: si scrive in **linguaggio naturale** "creami un circuito che faccia questo, questo e quest'altro, utilizzando l'Arduino" e il software prova a **generare lo schema**. È un prodotto commerciale a pagamento (circa 50 €/mese), quindi per ora resta lontano dall'uso quotidiano in laboratorio – ma è un esempio di come, essendo la netlist un semplice file di testo, in prospettiva l'intero processo di disegno potrebbe essere **automatizzato**.

2.1 Il flusso completo: schema → netlist → PCB



Figura 1: Il flusso di progettazione: dallo schema elettrico, tramite la netlist (elenco delle connessioni), si arriva al circuito stampato vero e proprio.

Dal **disegno schematico**, KiCad genera la **netlist**; grazie a questa lista di **connessioni** si può produrre il **layout** (il PCB vero e proprio). Nelle versioni recenti di KiCad questo passaggio è più diretto, ma concettualmente restano tre fasi distinte.

3 Creare un nuovo progetto

Per creare un **nuovo progetto**: *File* → *Nuovo progetto*. Si sceglie una **cartella di salvataggio** (idealmente su Google Drive, per averla sempre disponibile) e si dà un nome al progetto. KiCad crea **due file** "figli": lo **schematico** e il **PCB** (inizialmente vuoto, da popolare).

Attenzione

Gestione dei file su Google Drive: aprire **sempre e solo** il **file di progetto** (estensione `.kicad_pro`), **mai** direttamente lo schematico o il PCB, altrimenti i **collegamenti** tra i file non funzionano. Se si scarica il progetto come **cartella compressa** (`.zip`), va **sempre estratta per intero** prima di lavorarci: aprire i file dentro lo zip senza estrarli porta quasi sempre a **perdere il lavoro** al momento del salvataggio.

4 Disegnare un componente: il resistore

Per posizionare un **resistore**: si preme il tasto **A**, si sceglie il **simbolo** (rettangolare, non a zig-zag), lo si posiziona sul foglio. KiCad assegna automaticamente un **nome progressivo** (R1, R2, ...) e permette di impostare il valore (es. 250 Ω) con doppio clic.

Nota

Cliccando su un singolo **pin** di un componente (non sul corpo), viene evidenziato **solo quel piedino** e KiCad mostra il suo numero – utile per identificare correttamente i collegamenti anche su componenti con molti piedini (es. 40 pin).

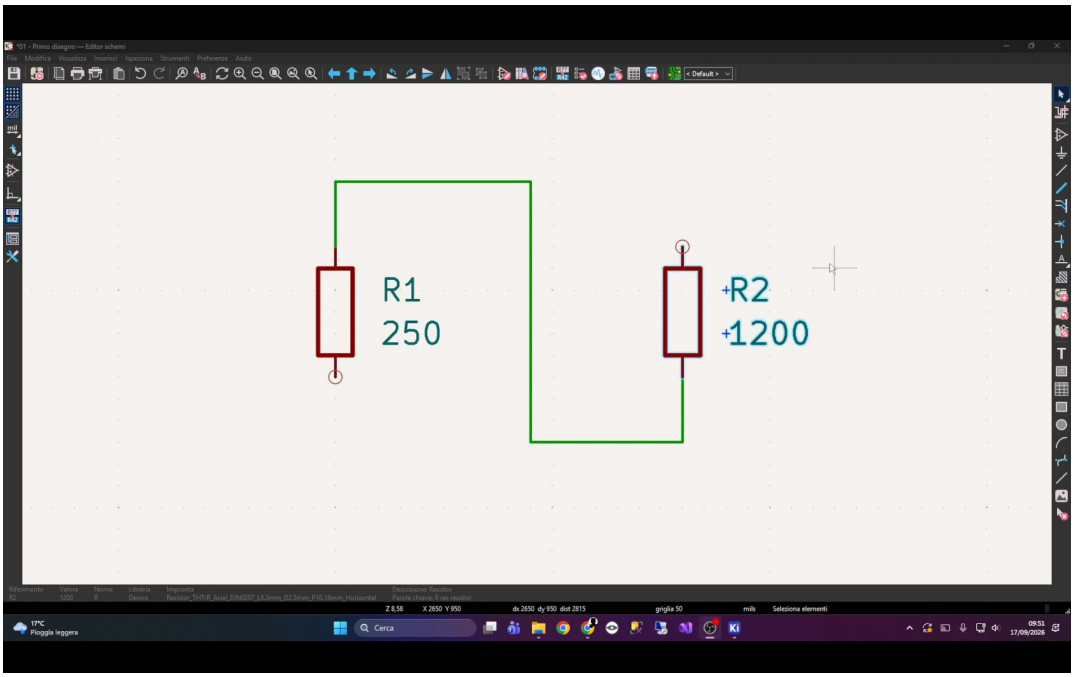


Figura 2: Schermata reale dell'editor schematico di KiCad: due resistori (R1 = 250 Ω, R2 = 1200 Ω) collegati tramite un filo (la linea verde rappresenta la connessione elettrica tra i pin).

5 Dal simbolo al footprint

Dopo aver disegnato lo schema, ogni componente va **associato** al suo **footprint**: l'**impronta fisica** che occuperà una volta saldato sul circuito stampato. Si seleziona il componente e si preme il tasto **F**.

Definizione

Il **footprint** è la **vista dall'alto** dell'**ingombro** e dei **punti di saldatura** di un componente reale – non un disegno artistico, ma un **dato tecnico** usato per generare il circuito stampato.



PTH: fori passanti **SMD: piazzole piatte**

Figura 3: Confronto tra footprint: a sinistra un componente PTH con fori passanti da saldare, a destra un componente SMD con piazzole piatte su cui appoggiarsi.

Definizione

PTH (Pin/Plated Through Hole, "a foro passante"): il piedino del componente **attraversa un foro** nella scheda, per essere saldato dal **lato opposto**.
SMD (Surface Mounting Device, "a montaggio superficiale"): il componente si **appoggia** direttamente su **piazzole piatte**, **senza fori**.

I componenti **SMD** sono generalmente **più piccoli** e **più economici** da produrre in serie (richiedono meno tempo di lavorazione); i componenti **PTH** sono **più semplici** da saldare a mano e **più ro-**

busti meccanicamente. In libreria PTH si trova a volte anche sotto la sigla **THT** (*Through-Hole Technology*): **stessa cosa**, sigla diversa.

Esempio

Il **codice** di un footprint per un **resistore assiale** (es. `R_Axial_DIN0207`) contiene informazioni codificate: **lunghezza** del corpo (es. `L6.3mm`), **diametro** (`D2.5mm`), **passo** tra i fori (es. `P15.24mm`). Si **confrontano** queste misure col **componente reale** in mano per scegliere il footprint più adatto: nell'esempio fatto a lezione, un resistore da 6,3 mm di lunghezza e 2,5 mm di diametro non aveva in libreria un passo esatto di 38 mm, e la scelta è ricaduta su un **passo più piccolo** (10 mm), sufficiente per come si intendeva **piegare i terminali**. Se non esiste una corrispondenza esatta, meglio scegliere una versione **leggermente più grande** che più piccola, per essere sicuri che il componente ci stia fisicamente (tipico il caso dei **condensatori**, cilindri piuttosto voluminosi).

Nota

Nella barra di ricerca della **libreria** si può usare l'**asterisco** (`*`) come **carattere jolly** ("come la matta nel gioco delle carte"), per filtrare i componenti: sostituisce una qualunque sequenza di caratteri. Ad esempio, cercando `M*O` si troverebbero "Mauro", "Maurizio", "Marco", "Moreno" – qualsiasi parola che **inizia per M** e **finisce per O**. Esiste anche il **punto interrogativo** (`?`), che sostituisce invece un **singolo carattere** soltanto: `M?O` troverebbe "MIO" o "MAO" ma non "MADDO" (troppi caratteri) né "Mauro" (che ne ha quattro in mezzo, per cui servirebbe `M????O` o l'asterisco). È lo stesso meccanismo di ricerca con **wildcard** usato in DOS/Windows (in altri linguaggi si usa il simbolo percentuale per lo stesso scopo). Applicato al nostro caso: `R*L6.3*` trova tutti i **resistori** con **lunghezza** 6.3 mm, a cui si può aggiungere un secondo filtro per il diametro (es. `*D2.5*`) per restringere ulteriormente la ricerca. Attenzione: la ricerca **non fa permutazioni**, quindi anche un **solo carattere sbagliato** nel filtro impedisce di trovare il componente.

Nota

Il **montaggio assiale** su circuito stampato è diventato lo standard fin dagli anni '90, perché una scheda di quel tipo occupa uno spazio noto e prevedibile. Prima dei PCB, i componenti si montavano "**in aria**", con **fili volanti**, fissati a una scatola solo per tenerli fermi (tipico dei vecchi **amplificatori valvolari**) oppure su una **turret board**: una tavola con perni metallici a cui i componenti venivano fissati e poi collegati con fili. Alcuni **amplificatori artigianali** si costruiscono ancora così oggi, perché secondo alcuni "suonano meglio" – una convinzione controversa, un po' come quella dei contatti placcati in oro.

6 Cosa ricordare di questa lezione

- **KiCad** separa lo **schema** elettrico (**connessioni logiche**) dal **PCB** (**disegno fisico**).
- Ogni componente ha **pin numerati**; le connessioni sono salvate in un file di testo chiamato **netlist**.
- Il flusso di lavoro è: **schema** → **netlist** → **PCB**.

- **Aprire sempre** il file `.kicad_pro`, **mai** schematico/PCB direttamente; **estrarre sempre** gli zip prima di lavorarci.
- Ogni componente ha un **footprint**: la sua **impronta fisica**, distinta dal **simbolo schematico**.
- Componenti **PTH** (foro passante) vs **SMD** (montaggio superficiale): differenze di **dimensione** e **modalità di saldatura**.

Esempio

Da fare per la prossima lezione: osserva un **componente elettronico** che hai in casa o a scuola e prova a capire se è **PTH** o **SMD**, e perché.